

2026-2 컴퓨터 비전

기초 코드 실습 1

담당교수 최재영
담당조교 한상훈

기초 코드 실습 목표

1. 기본적인 Python 환경 구축
2. 코딩 과제 대비 기초 Python 구현 학습

- 1주차

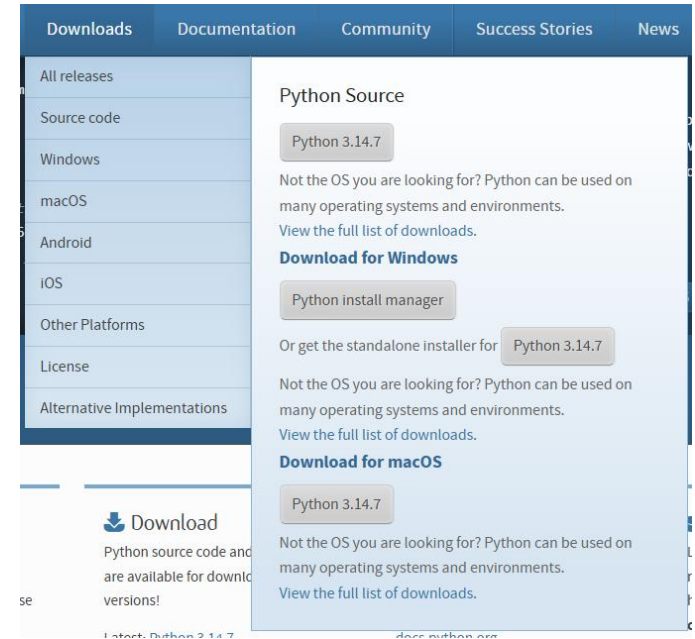
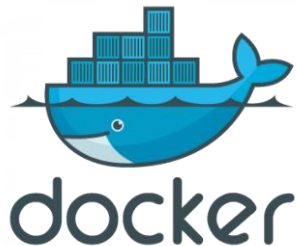
- Python 기초 : list 변환
- NumPy 기초 : list 변환, 이미지 변환
- OpenCV 기초 : 이미지 변환
- Image masking
- OpenCV 라이브러리 함수들

- 2주차

- OpenCV Features
- Sklearn 데이터셋 전처리

Python 설치

- Python 설치
 - 공식 URL 접속 후 설치: <https://www.python.org/>
 - Python 버전 3.10.x 권장
- OpenCV 설치
 - OpenCV 버전 4.10.x 권장
 - 특정 버전 설치 : `pip install opencv-python==4.10.0.84`
- Numpy 설치
 - Numpy 버전 1.26.4 권장
 - 특정 버전 설치 : `pip install numpy==1.26.4`
- 가상환경을 활용한 설치 권장 (virtualenv / anaconda / docker)



Python Data Structures

- List

```
myList = [1, 2, 3]
```

- Tuple

```
myTuple = (1, 2, 3)
```

- Dictionary

```
myDict = {'a':1, 'b':2, 'c':3}
```

- Python에서 여러 개의 데이터를 다루는 자료형들 – List, Tuple, Dictionary
 - 선언하는 방법에 따라 자료형이 달라지는 것을 볼 수 있음
 - List: [...], Tuple: (...), Dictionary: { 'key1':value1, 'key2':value2, ... }

Python Data Structures

- List

```
>>> myList = [1, 2, 3]
>>> myList[0]
1
>>> myList[1]
2
>>> myList[2]
3
>>> myList[0] = 100 (OK)
>>> myList[0]
100
```

[1, 2, 3]
↓
[100, 2, 3]

- Tuple

```
>>> myTuple = (1, 2, 3)
>>> myTuple[0]
1
>>> myTuple[1]
2
>>> myTuple[2]
3
>>> myTuple[0] = 100 (ERROR)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
```

- List는 여러 값을 순서대로 저장하고 각 요소를 수정 가능 (mutable)
- Tuple은 List처럼 값을 읽을 수는 있지만 요소 수정은 불가능 (immutable)
 - Tuple의 각 요소를 바꿀 수는 없지만, 변수에 새로운 Tuple을 다시 할당 가능

Python Data Structures

- Dictionary

```
>>> myDict = {'a':1, 'b':2, 'c':3}
>>> myDict['a']
1
>>> myDict['b']
2
>>> myDict['c']
3
>>> myDict['a'] = 100
>>> myDict
{'a': 100, 'b': 2, 'c': 3}
```

- Dictionary는 key - value 쌍으로 데이터를 저장하며, key를 통해 데이터에 접근
 - List, Tuple의 경우 Index를 통해서 데이터에 접근함.
- 각 요소를 수정할 수 있는 mutable 자료형

List Functions

```
mylist = [1,2,3]

mylist2 = [] # list init
mylist2.append(1) # [1]
mylist2.append(2) # [1,2]
mylist2.append(3) # [1,2,3]

print(mylist, mylist2)
```

[1, 2, 3] [1, 2, 3]

- **append(x)**
list 맨 끝에 x를 추가

```
mylist = [1]

mylist2 = [1,2,3,1] # list init
mylist2.remove(3) # [1,2,1]
mylist2.remove(2) # [1,1]
mylist2.remove(1) # [1] not []

print(mylist, mylist2)
```

[1] [1]

- **remove(x)**
list에서 x 값을 하나 삭제

```
mylist = [1]

mylist2 = [1,2,3] # list init
mylist2.pop() # [1,2]
mylist2.pop() # [1]

print(mylist, mylist2)
```

[1] [1]

- **pop()**
list에서 마지막 요소 삭제
- **pop(i)**
list에서 i번째 요소 삭제

List – Indexing & Slicing

```
Idx: 0  1  2  3
>>> mylist = [1, 2, 3, 4]
>>> mylist[0]
1
>>> mylist[1]
2
>>> mystring = 'abcd'
>>> mystring[0]
'a'
>>> mystring[1]
'b'
```

List indexing

```
Idx: 0  1  2  3  4
>>> mylist = [1, 2, 3, 4, 5]
>>> mylist[1:3]
[2, 3]
>>> mylist[2:5]
[3, 4, 5]
>>> mystring = 'vision'
>>> mystring[1:4]
'isi'
```

List slicing

- Index는 0부터 시작함.
- `list[i]` : i번째 요소 값 가져오기 (Indexing)
- `list[start:end]` : start번째 ~ (end-1) 번째의 값들 가져오기 (Slicing)

Scalar, Vector, Matrix, Tensor

7

$$\begin{bmatrix} 15 \\ -3 \\ \vdots \\ 7 \end{bmatrix}$$

$$\begin{bmatrix} 15 & 1 & \dots & 11 \\ -3 & 8 & \dots & 9 \\ \vdots & \vdots & \ddots & \vdots \\ 7 & 9 & \dots & 5 \end{bmatrix}$$

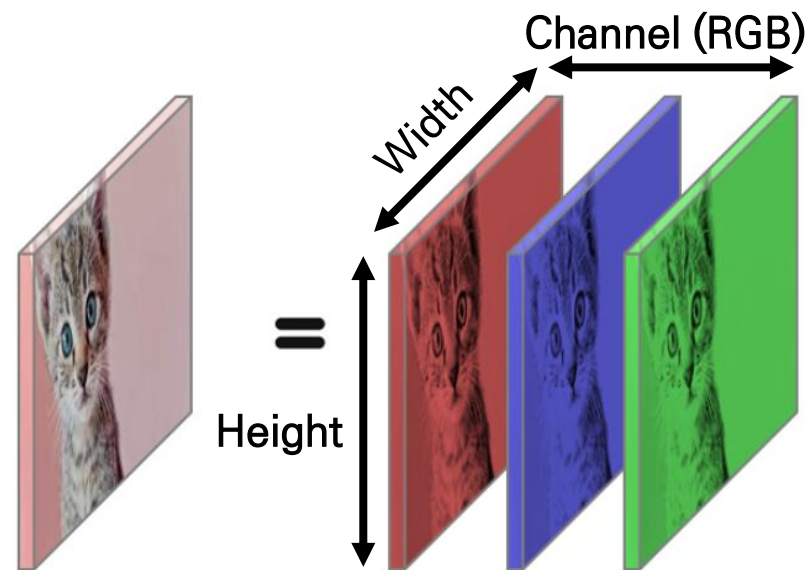
$$\left\{ \begin{bmatrix} 15 & 1 & \dots & 11 \\ -3 & 8 & \dots & 9 \\ \vdots & \vdots & \ddots & \vdots \\ 7 & 9 & \dots & 5 \end{bmatrix} \right\}$$

스칼라
0차원

벡터
1차원

행렬
2차원

3차원 텐서
3차원



실제 이미지는 3차원 텐서로 표현

Numpy

- 벡터 및 행렬 연산에 편리한 기능을 제공하는 라이브러리
- Pandas, OpenCV, PyTorch... 의 기반 라이브러리

```
import numpy as np

mylist = [1,2,3]
myarray = np.array(mylist) #basic data unit

print(type(mylist)) # list
print(type(myarray)) # ndarray

print(myarray.shape) # check shape
print(myarray.dtype) # check data type

myarray2 = np.array(mylist, dtype = np.uint8) # specify data type
print(myarray2.dtype)
print((myarray2.astype(np.int8)).dtype) # change data type
```

```
<class 'list'>
<class 'numpy.ndarray'>
(3,)
int32
uint8
int8
```

NumPy 기본 예시

Numpy – Initialization

- NumPy array – N차원의 데이터를 저장. 다양한 방법으로 초기화 가능

np.zeros

```
print(np.zeros(10))  
print(np.zeros((3,5)))
```

```
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]  
[[0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0.]]
```

np.ones

```
print(np.ones(3))  
print(np.ones((2,3,3)))
```

```
[1. 1. 1.]  
[[[1. 1. 1.]  
   [1. 1. 1.]  
   [1. 1. 1.]]  
 [[1. 1. 1.]  
   [1. 1. 1.]  
   [1. 1. 1.]]]
```

np.arange, full

```
print(np.arange(3))  
print(np.arange(3,10))
```

```
[0 1 2]  
[3 4 5 6 7 8 9]
```

```
: #init full  
print(np.full((3,2),1))
```

```
[[1 1]  
 [1 1]  
 [1 1]]
```

shape value

np.random.rand

```
print(np.random.rand(3,3,4))
```

```
[[[0.44271133 0.29626794 0.41089846 0.3064163 ]  
  [0.42928754 0.55529644 0.98317838 0.3824213 ]  
  [0.34121595 0.37673191 0.54378304 0.31438084]]  
  
 [[0.38973088 0.78485194 0.66848924 0.77456111]  
  [0.04333284 0.63142971 0.15119697 0.79550842]  
  [0.43995525 0.86339722 0.29779624 0.41829296]]  
  
 [[0.73041168 0.64816277 0.78612294 0.8866486 ]  
  [0.91835649 0.92070584 0.76462882 0.82035006]  
  [0.82540981 0.27335881 0.7031834 0.00157336]]]
```

np.random.randint

```
print(np.random.randint(0,10, (3,3,4)))
```

```
[[[2 6 9 1]  
  [0 3 3 3]  
  [0 7 1 5]]  
  
 [[4 4 2 3]  
  [5 3 0 8]  
  [3 6 5 2]]  
  
 [[4 2 0 8]  
  [9 3 1 1]  
  [8 1 9 1]]]
```

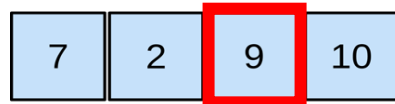
np.random.randn

```
: print(np.random.randn(3,3,4)) # normal distribution
```

```
[[[ 0.31872579 0.22880617 1.30821783 0.46690768]  
  [ 0.10283885 -1.19910601 -0.16199321 0.2003674 ]  
  [-1.51038025 0.89345824 -1.38378566 1.11221919]]  
  
 [[ 1.02064927 -0.81154136 -0.19491351 -0.4799937 ]  
  [-0.63563444 1.33441645 -0.62340852 -0.25192448]  
  [-0.48262618 -0.06644451 1.67294596 -1.27988334]]  
  
 [[-1.09885304 0.36852243 -0.03255591 0.09487077]  
  [ 1.3921729 1.51277877 -0.89601284 -0.79357861]  
  [ 1.49134765 -0.7367949 1.45628968 0.06219378]]]
```

Numpy - Array

1D array



axis 0 →

shape: (4,)

`array[2] = 9`

2D array

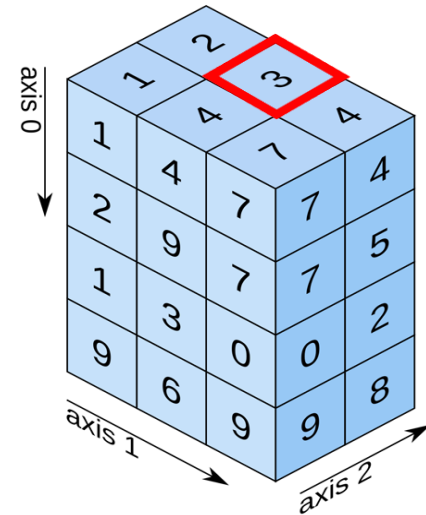


axis 1 →

shape: (2, 3)

`array[1,1] = 0.1`

3D array



shape: (4, 3, 2)

`array[0,1,1] = 3`

- NumPy Array에 접근할 때는 axis (dimension) 기준으로 접근

Numpy – Indexing & Slicing

```
example = np.arange(0,10)
print(example)
```

```
[0 1 2 3 4 5 6 7 8 9]
```

```
print(example[0]) # 0
print(example[-1]) # last element
```

```
0
```

```
9
```

```
print(example[2:5]) # 2 to 4
print(example[:]) # everything
```

```
[2 3 4]
```

```
[0 1 2 3 4 5 6 7 8 9]
```

1차원 Array Indexing & Slicing

```
example = np.arange(25).reshape(5,5)
print(example)
```

```
[[ 0  1  2  3  4]
 [ 5  6  7  8  9]
 [10 11 12 13 14]
 [15 16 17 18 19]
 [20 21 22 23 24]]
```

```
print(example[0]) # 0th row
```

```
[0 1 2 3 4]
```

```
print(example[2,4]) # 2th row 4th column element
```

```
14
```

```
print(example[0:2, 1:3]) # from 0 to 1 row, 1 to 2 column
```

```
[[1 2]
 [6 7]]
```

2차원 Array Indexing & Slicing

- List의 Indexing & Slicing 하는 방법과 유사
- NumPy Array 또한 Index 0부터 시작

Numpy – Indexing & Slicing



- PIL, OpenCV 등 파이썬 라이브러리로 이미지를 열었을 때 좌표는?

Numpy – Image

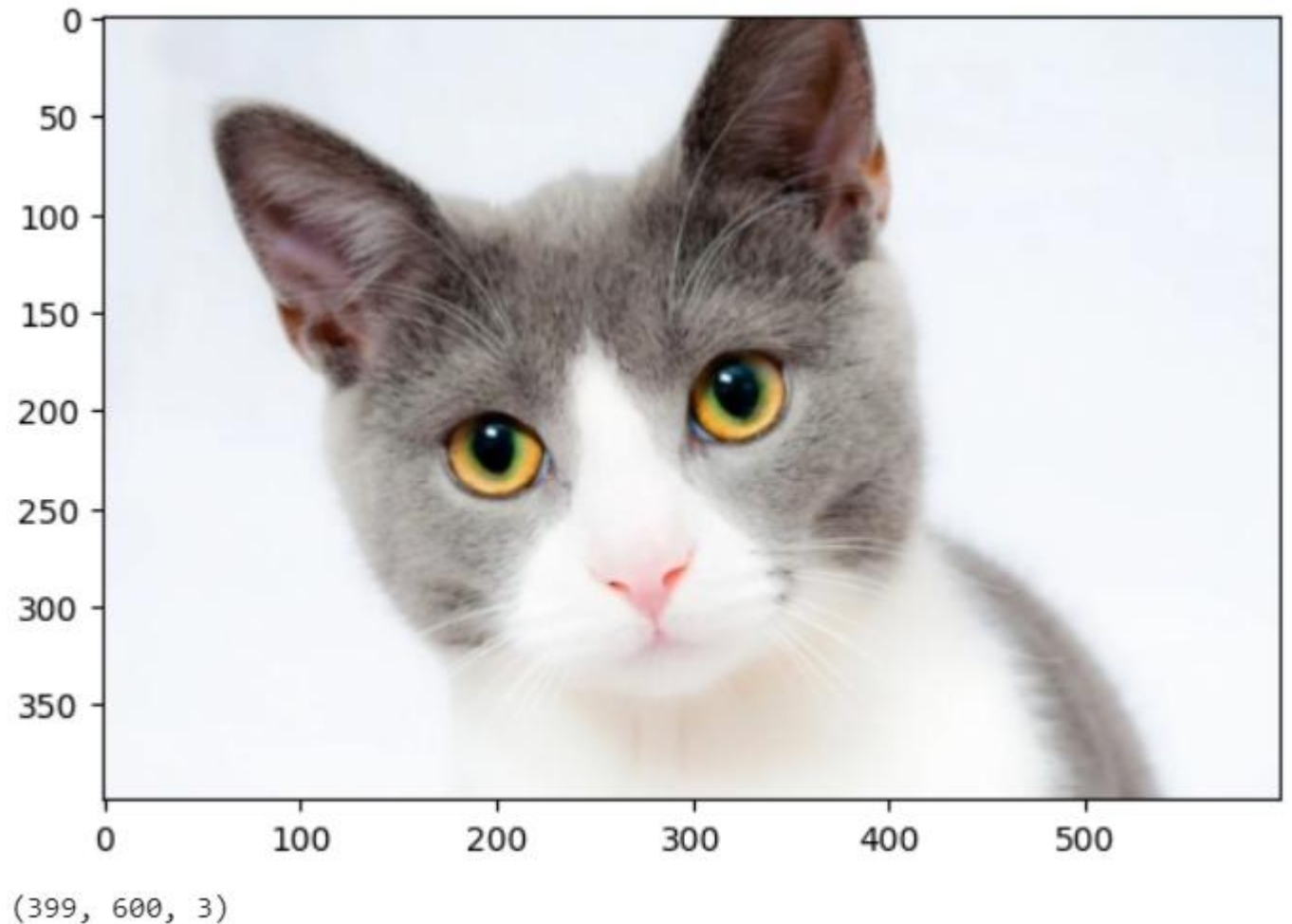
```
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt

path = 'Desktop/cat.jpg' # input your image path

image_pil = Image.open(path)
image = np.array(image_pil)

plt.imshow(image)
plt.show()
print(image.shape)
```

샘플 코드



Numpy – Image Tensor Slicing

```
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt

path = 'Desktop/cat.jpg' # input your image path

image_pil = Image.open(path)
image = np.array(image_pil)

plt.imshow(image)
plt.show()
print(image.shape)
```

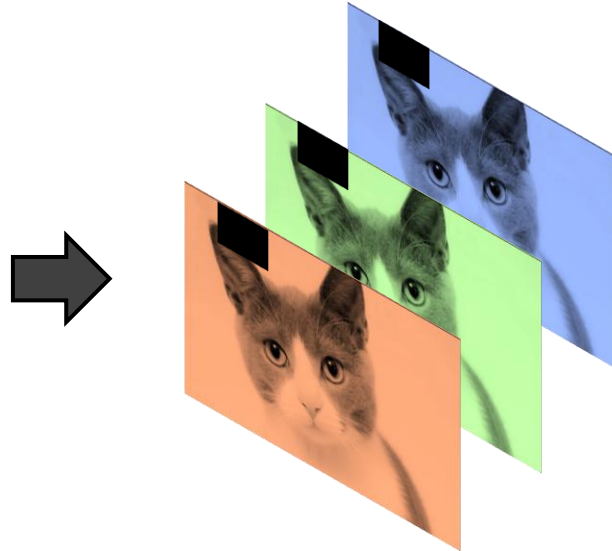


`Image[0:50, 100: 200] = 0`

이미지의 세로 0~50픽셀, 가로 100~200px픽셀을 검은색(0, 0, 0)으로 만들

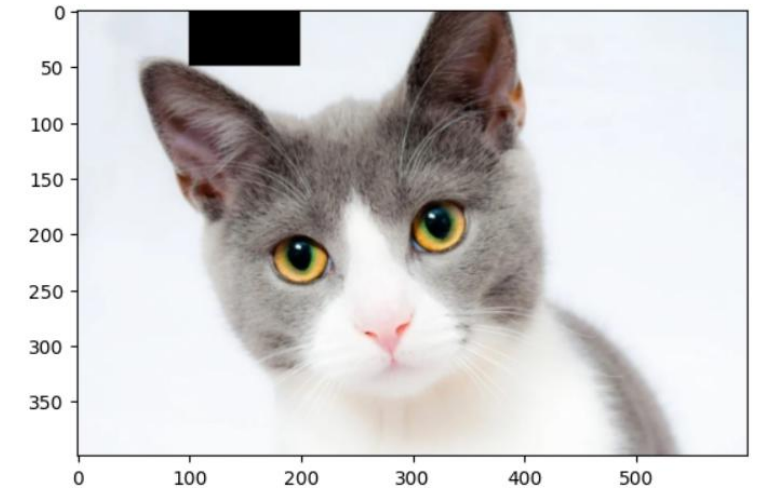
→ `Image[0:50, 100: 200, :] = 0`

→ `Image[0:50, 100: 200, 0:3] = 0`



```
image[0:50,100:200] = [0,0,0]
plt.imshow(image)
plt.show
```

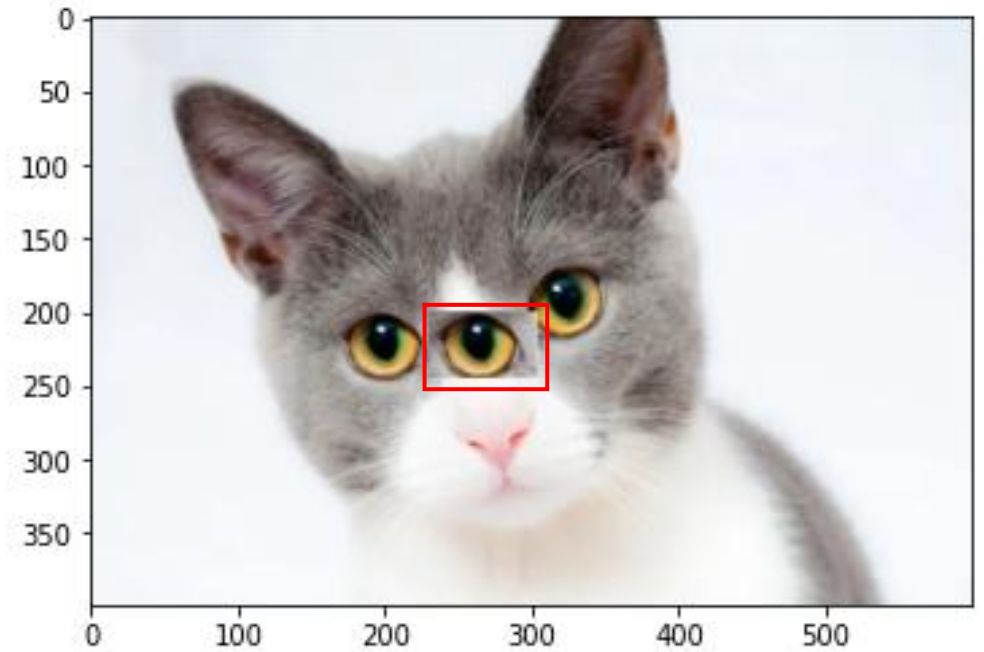
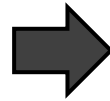
<function matplotlib.pyplot.show(close=None, block=None)>



- 실제 이미지를 다룰 때는 (image_height, image_width, 3) 형태의 3차원 데이터로 표현됨

Numpy – Image Tensor Slicing

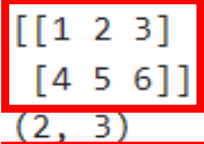
```
eye = image[200:245, 165:240].copy()  
image[200:245, 230:305] = eye  
  
plt.imshow(image)  
plt.show()
```

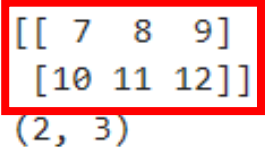


- Slicing을 통해서 이미지의 영역 지정 가능
 - Region = Image[y1:y2, x1:x2]
- Masking, Draw등에서 다양하게 활용 가능

Numpy – Array 연산

```
arr1 = np.array([[1,2,3],[4,5,6]])  
arr2 = np.array([[7,8,9],[10,11,12]])  
print(arr1)  
print(arr1.shape)  
print(arr2)  
print(arr2.shape)
```

arr1 
[[1 2 3]
 [4 5 6]]
(2, 3)

arr2 
[[7 8 9]
 [10 11 12]]
(2, 3)

```
print(arr1 + arr2)
```

```
[[ 8 10 12]  
 [14 16 18]]
```

```
print(arr1 * arr2)
```

```
[[ 7 16 27]  
 [40 55 72]]
```

```
print(arr1 - arr2)
```

```
[[ -6 -6 -6]  
 [ -6 -6 -6]]
```

Array 연산

- Array끼리 연산(사칙연산 등)을 수행할 수 있음. 이 때 각 element의 위치에서 수행됨
- Array끼리 연산할 때는 각 array의 차원(shape)이 일치해야 함. 알맞지 않은 경우 error 발생

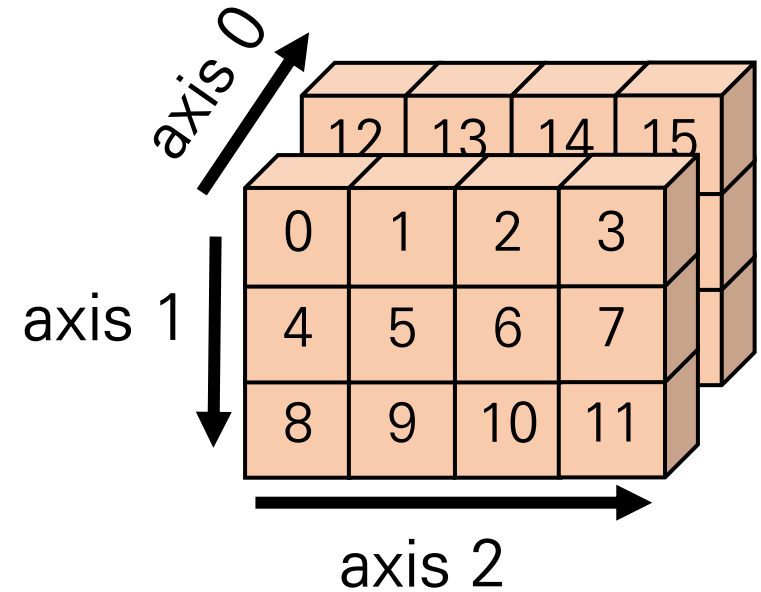
Numpy – Array 함수 (sum)

```
print("example : ")
print(example) #shape (2,3,4)
print("all sum :", np.sum(example))
print("axis 0 :", np.sum(example,axis = 0)) # summing by axis 0 (0)
print(np.sum(example, axis = 0))
print("axis 1 :",) # summing by axis 1 (1)
print( np.sum(example,axis = 1))
print("axis 2 :",) # summing by axis 2 (2)
print( np.sum(example,axis = 2))
```

```
example :
[[[ 0  1  2  3]
  [ 4  5  6  7]
  [ 8  9 10 11]]

 [[12 13 14 15]
  [16 17 18 19]
  [20 21 22 23]]]
all sum : 276
```

axis 0 :	[[12 14 16 18] [20 22 24 26] [28 30 32 34]]
axis 1 :	[[12 15 18 21] [48 51 54 57]]
axis 2 :	[[6 22 38] [54 70 86]]



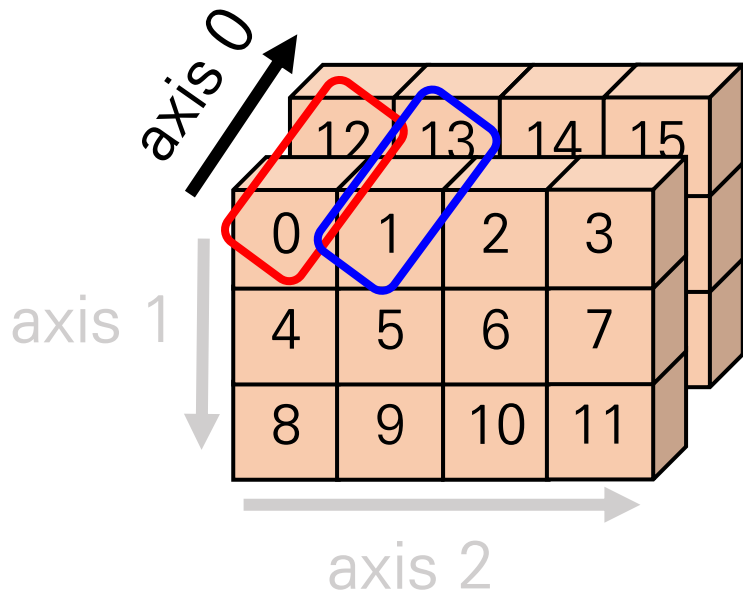
- np.sum, np.mean 등의 함수를 axis를 통해 개별 연산 가능

Numpy – Array 함수 (sum)

```
np.sum(example, axis = 0))
```

axis 0 :

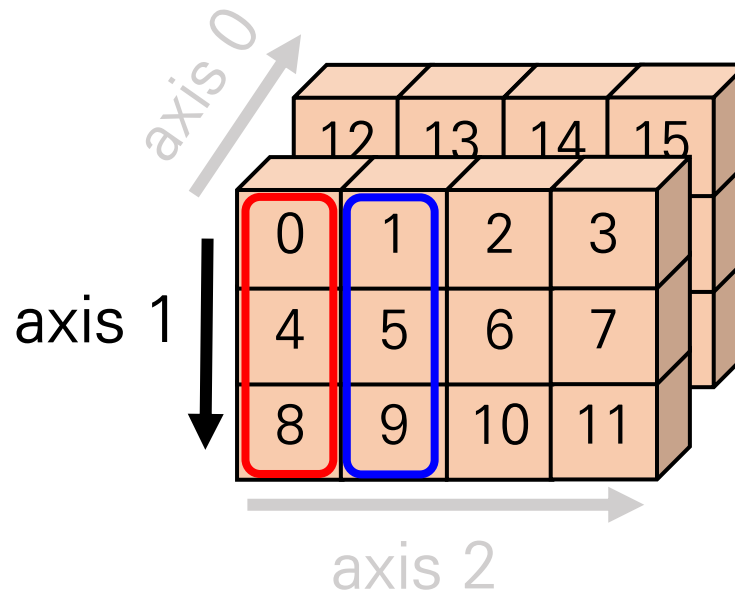
12	14	16	18
20	22	24	26
28	30	32	34



```
np.sum(example,axis = 1))
```

axis 1 :

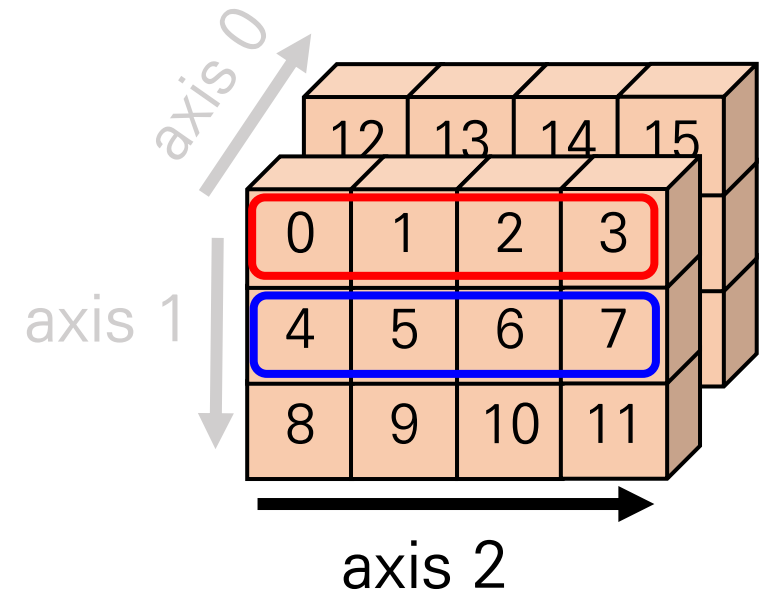
12	15	18	21
48	51	54	57



```
np.sum(example,axis = 2))
```

axis 2 :

6	22	38
54	70	86



Numpy – Array 함수 (reshape)

- np.reshape – array의 차원을 변경
- reshape 이전과 이후에 차원들의 요소 개수가 같아야함
e.g. (2, 3, 4) = (4, 3, 2) = 총 24개

```
print(example) #shape (2,3,4)
```

```
[[[ 0  1  2  3]
  [ 4  5  6  7]
  [ 8  9 10 11]]

 [[12 13 14 15]
  [16 17 18 19]
  [20 21 22 23]]]
```


Reshape

```
print(example.reshape(4,3,2))
```

```
[[[ 0  1]
  [ 2  3]
  [ 4  5]]

 [[ 6  7]
  [ 8  9]
  [10 11]]

 [[12 13]
  [14 15]
  [16 17]]

 [[18 19]
  [20 21]
  [22 23]]]
```

- reshape 함수에서 -1을 넣으면,
나머지 값에 맞춰 자동 설정

```
example2 = example.reshape(4,-1)
print(example2.shape)
```

(4, 6)

(4, -1) → (4,6)
인자의 개수가 24가 되도록
자동 설정

Numpy - Array 함수 (transpose)

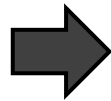
- `transpose(axes)` 혹은 `np.transpose(array, axes)` - array의 행과 열의 축을 재배열
- shape이 (2, 3, 4)인 array에 (1, 0, 2) 순서로 transpose → (3, 2, 4)로 변경

0 1 2
1 0 2

```
print(example) #shape (2,3,4)
```

```
[[[ 0  1  2  3]
   [ 4  5  6  7]
   [ 8  9 10 11]]

 [[12 13 14 15]
  [16 17 18 19]
  [20 21 22 23]]]
```



```
print(example.transpose(1,0,2))
print(example.transpose(1,0,2).shape)
```

```
[[[ 0  1  2  3]
   [12 13 14 15]]

 [[ 4  5  6  7]
   [16 17 18 19]]

 [[ 8  9 10 11]
   [20 21 22 23]]]
(3, 2, 4)
```

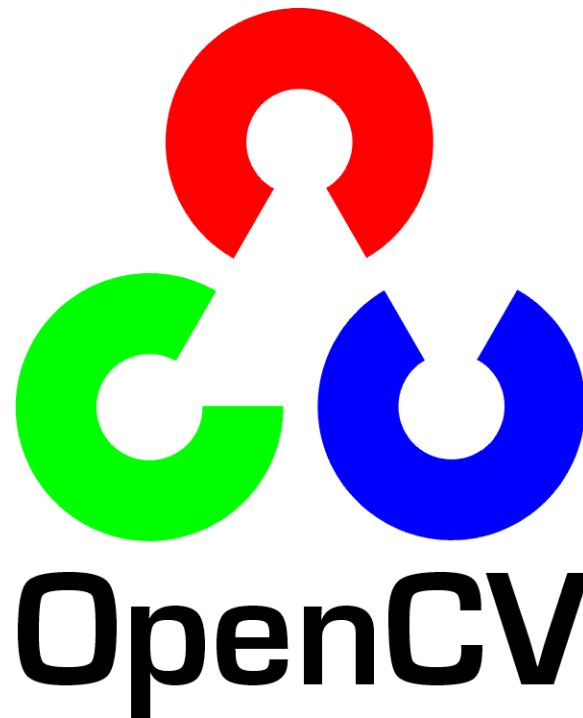
```
print(np.transpose(example, (2,1,0)))
print(np.transpose(example, (2,1,0)).shape)
```

```
[[[ 0 12]
   [ 4 16]
   [ 8 20]]

 [[ 1 13]
   [ 5 17]
   [ 9 21]]

 [[ 2 14]
   [ 6 18]
   [10 22]]

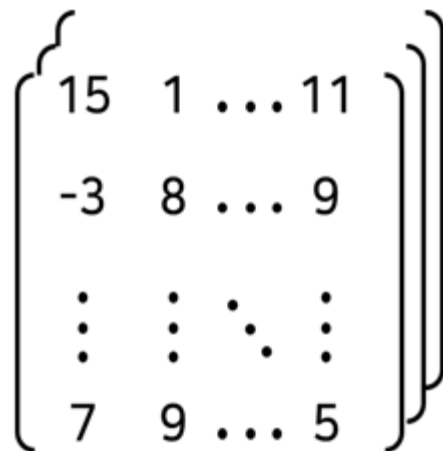
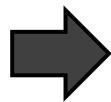
 [[ 3 15]
   [ 7 19]
   [11 23]]]
(4, 3, 2)
```



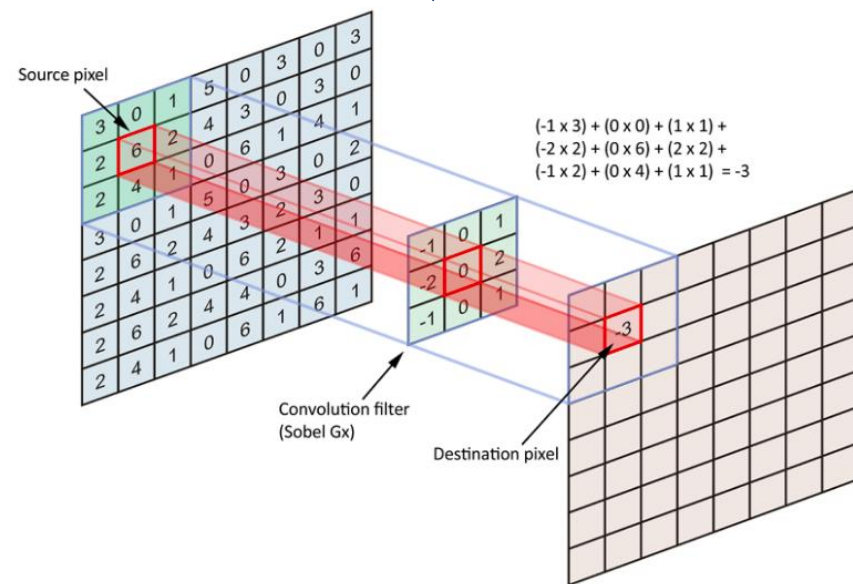
- Open Source Computer Vision
- C, C++, JAVA, Python...
- In python, NumPy + OpenCV is awesome!
- Image warping, filtering, binize, edge detect, pattern recognition, ROI...

What is an Image?

- 컴퓨터는 RGB 이미지를 3차원 텐서로 변환하여 처리
- 변환된 3차원 텐서를 바탕으로 여러 function 적용 가능
- OpenCV를 이용하여 이와 같은 작업을 쉽게 처리 가능



3차원 텐서



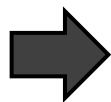
Function $f(\text{image})$

OpenCV – imread 함수

- 이미지를 불러올 때: cv2.imread()
- cv2.imread로 불러온 이미지를 matplotlib로 띄우면 이미지 색이 다름

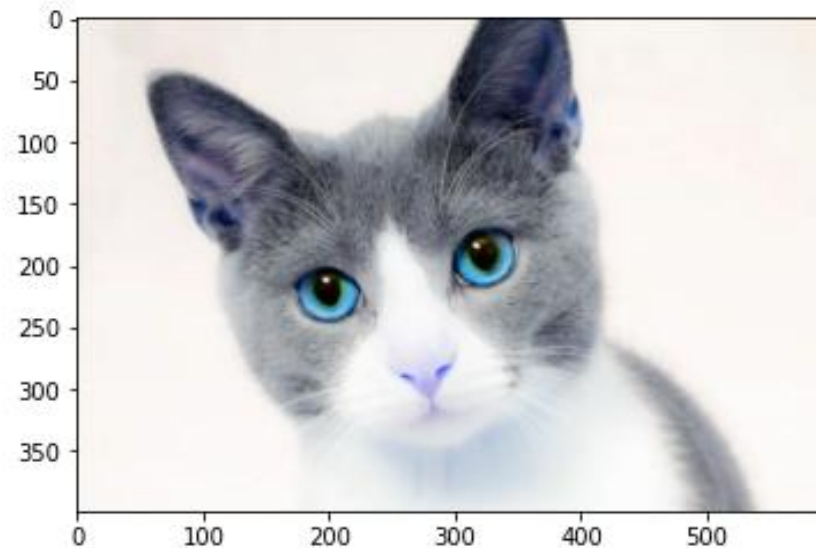


원본 이미지



```
import cv2  
  
imgcv = cv2.imread('cat.jpg')  
  
plt.imshow(imgcv)
```

<matplotlib.image.AxesImage at 0x1f8f5dd3d90>



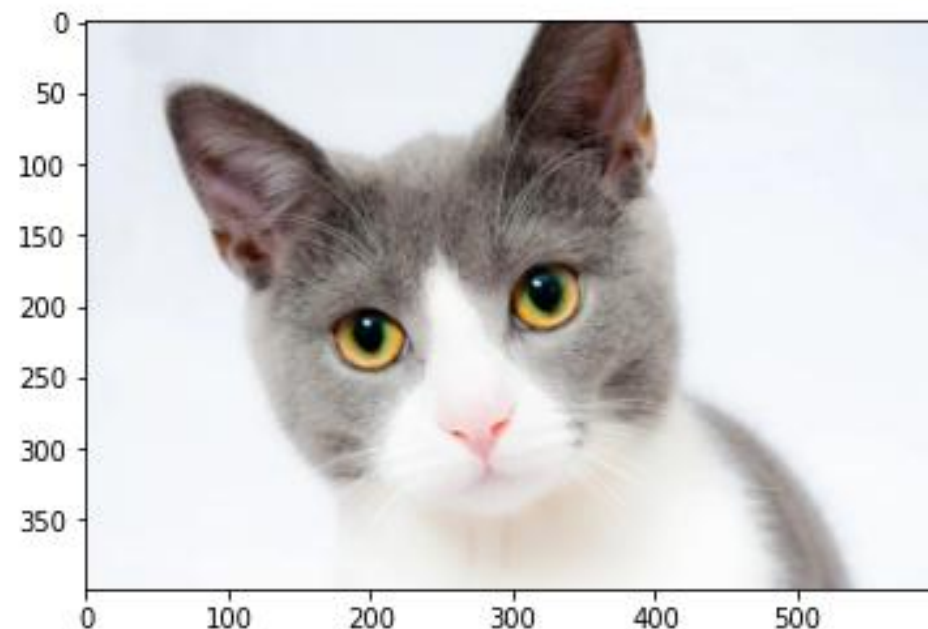
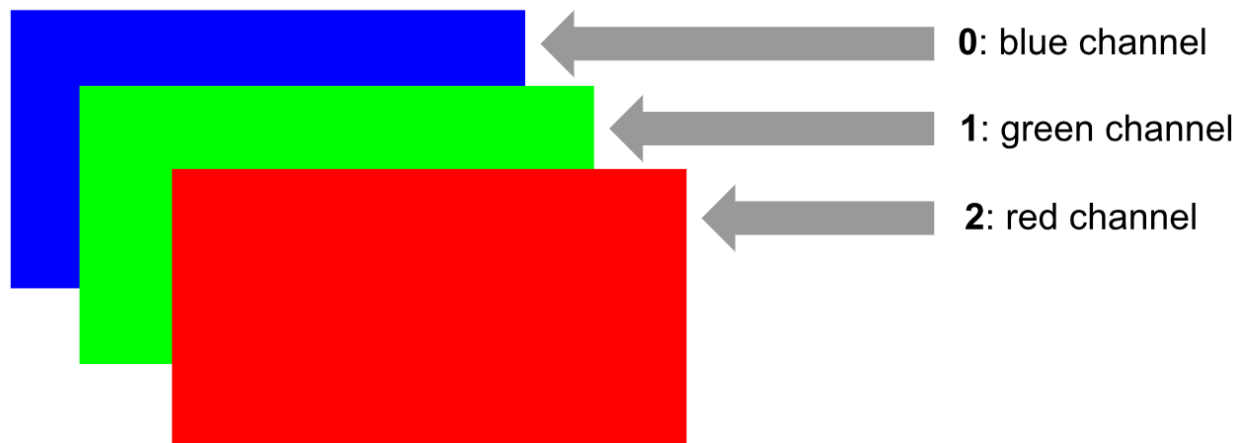
OpenCV – Channel order

- Matplotlib: R-G-B / OpenCV: B-G-R 채널 순서

→ `cv2.cvtColor()`, `cv2.COLOR_BGR2RGB` 인자로 채널 순서 변환
(미적용 시 과제 감점 주의!)

```
imgcv_rgb = cv2.cvtColor(imgcv, cv2.COLOR_BGR2RGB)  
plt.imshow(imgcv_rgb)
```

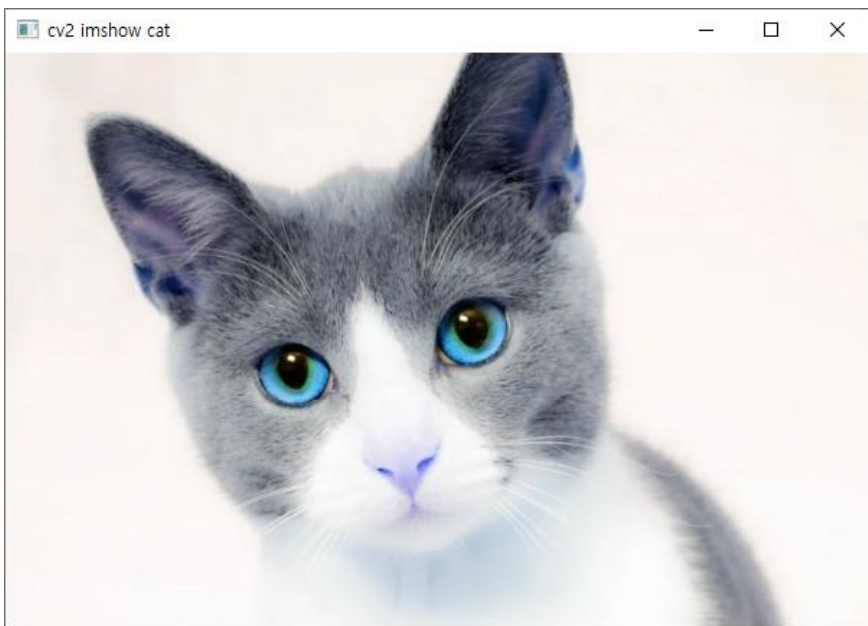
<matplotlib.image.AxesImage at 0x1f8f5e4d430>



OpenCV – imshow 함수

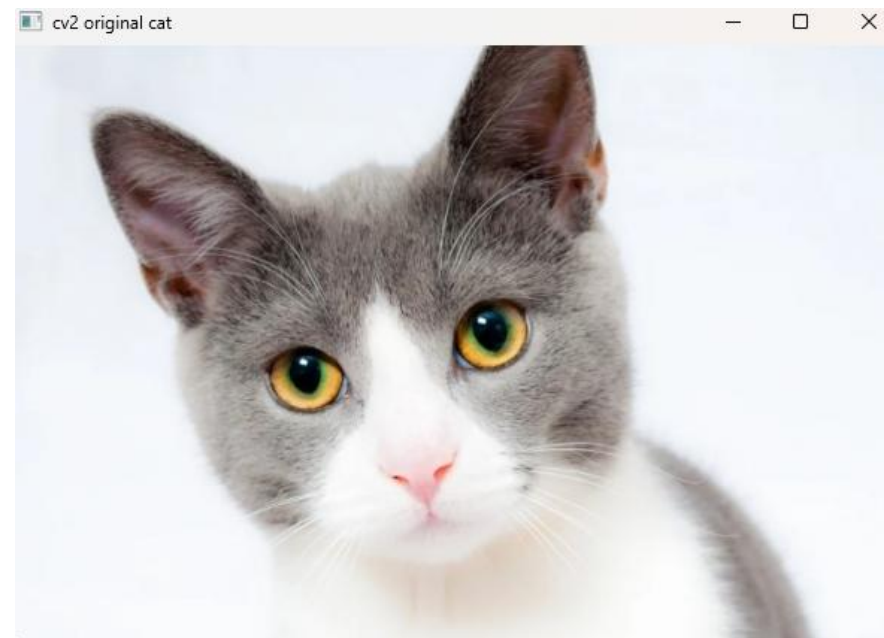
- cv2.imshow('title', img) 함수를 이용하여 이미지 확인 가능
- 주의사항: cv2.imshow 함수에는 채널 변환 X (BGR을 그대로 사용하기 때문)
matplotlib의 plt.imshow 를 사용할 때에만 채널 변환

```
cv2.imshow('cv2 imshow cat',imgcv_rgb)  
cv2.waitKey(0)  
cv2.destroyAllWindows()
```



BGR2RGB cv2.read -> cv2.imshow

```
cv2.imshow('cv2 imshow cat',imgcv)  
cv2.waitKey(0)  
cv2.destroyAllWindows()
```



cv2.read -> cv2.imshow

OpenCV – imwrite 함수

- 파일 저장할 때에는 cv2.imwrite('filename', image)
- imread, imwrite 둘다 경로 정확하게 적어야 imwrite 작동!

```
cv2.imwrite('wierd_cat.jpg', imgcv_rgb)
```



OpenCV – Image Binarization

- Image를 기준값 (threshold) 에 따라 흑 또는 백으로 분리(이진화)하기 위해 사용
- 1. 먼저 Image를 grayscale로 변환 RGB 3채널 이미지 → 밝기 정보만 있는 1채널 이미지
→ `cv2.imread('cat.jpg', cv2.IMREAD_GRAYSCALE)`
- 2. grayscale 이미지에 기준값 (`cv2.threshold`, `cv2.adaptiveThreshold`) 적용

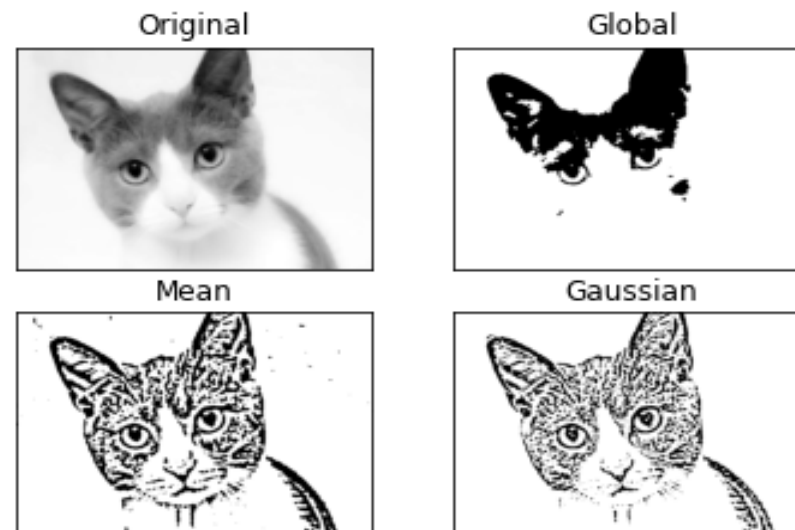
```
ret, th1 = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)
th2 = cv2.adaptiveThreshold(img, 255, cv2.ADAPTIVE_THRESH_MEAN_C, cv2.THRESH_BINARY, 15, 2)
th3 = cv2.adaptiveThreshold(img, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 15, 2)

titles = ['Original', 'Global', 'Mean', 'Gaussian']

images = [img, th1, th2, th3]

for i in range(4):
    plt.subplot(2, 2, i+1), plt.imshow(images[i], 'gray')
    plt.title(titles[i])
    plt.xticks([], plt.yticks([]))

plt.show()
```



OpenCV Image Transformations

- 이미지 사이즈 변경하기
- `cv2.resize(img, dsize, fx, fy, interpolation)`
 - 이미지 축소 시엔 interpolation 인자에 `cv2.INTER_AREA`, 확대 시엔 `cv2.INTER_CUBIC` 알고리즘 사용 권장

```
img = imgcv.copy()

height, width = img.shape[:2]

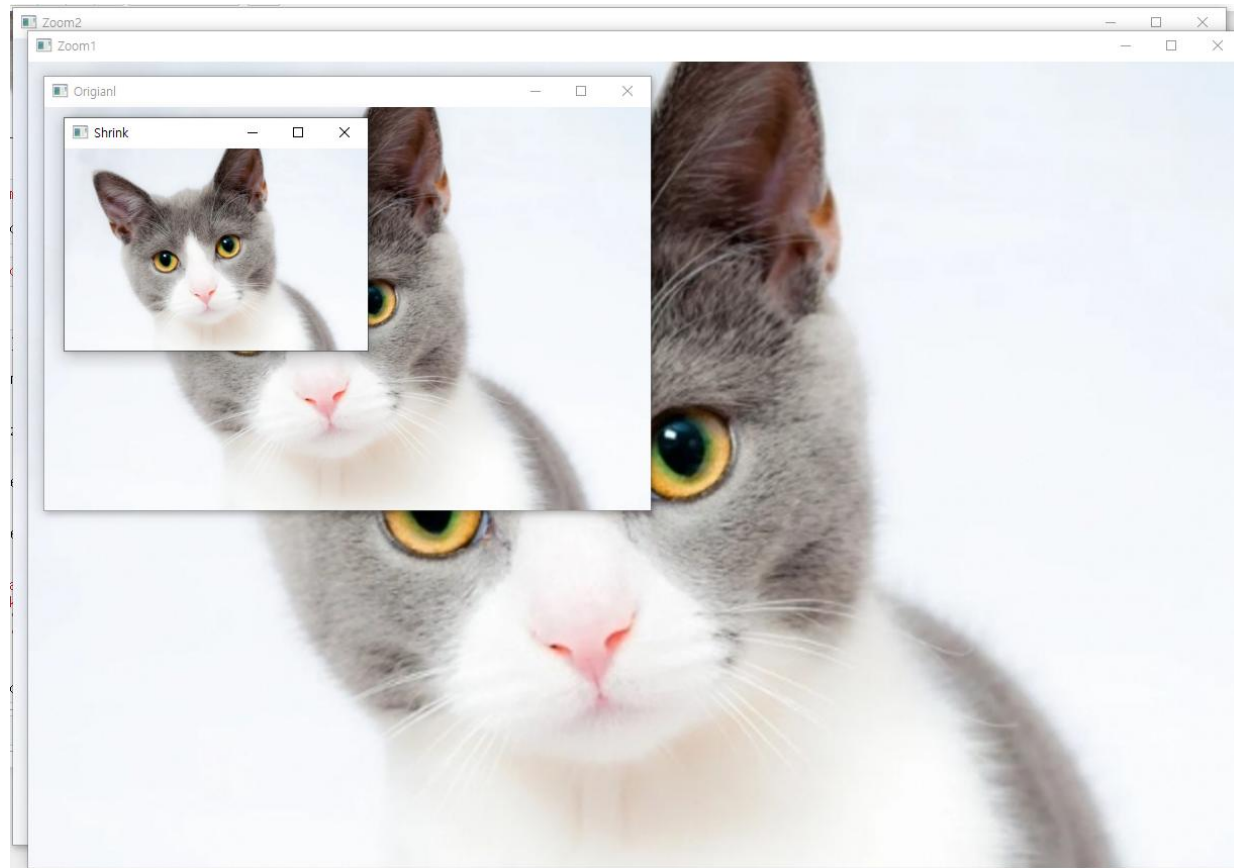
# 이미지 축소
shrink = cv2.resize(img, None, fx=0.5, fy=0.5, interpolation=cv2.INTER_AREA)

# Manual Size지정
zoom1 = cv2.resize(img, (width*2, height*2), interpolation=cv2.INTER_CUBIC)

# 배수 Size지정
zoom2 = cv2.resize(img, None, fx=2, fy=2, interpolation=cv2.INTER_CUBIC)

cv2.imshow('Original', img)
cv2.imshow('Shrink', shrink)
cv2.imshow('Zoom1', zoom1)
cv2.imshow('Zoom2', zoom2)

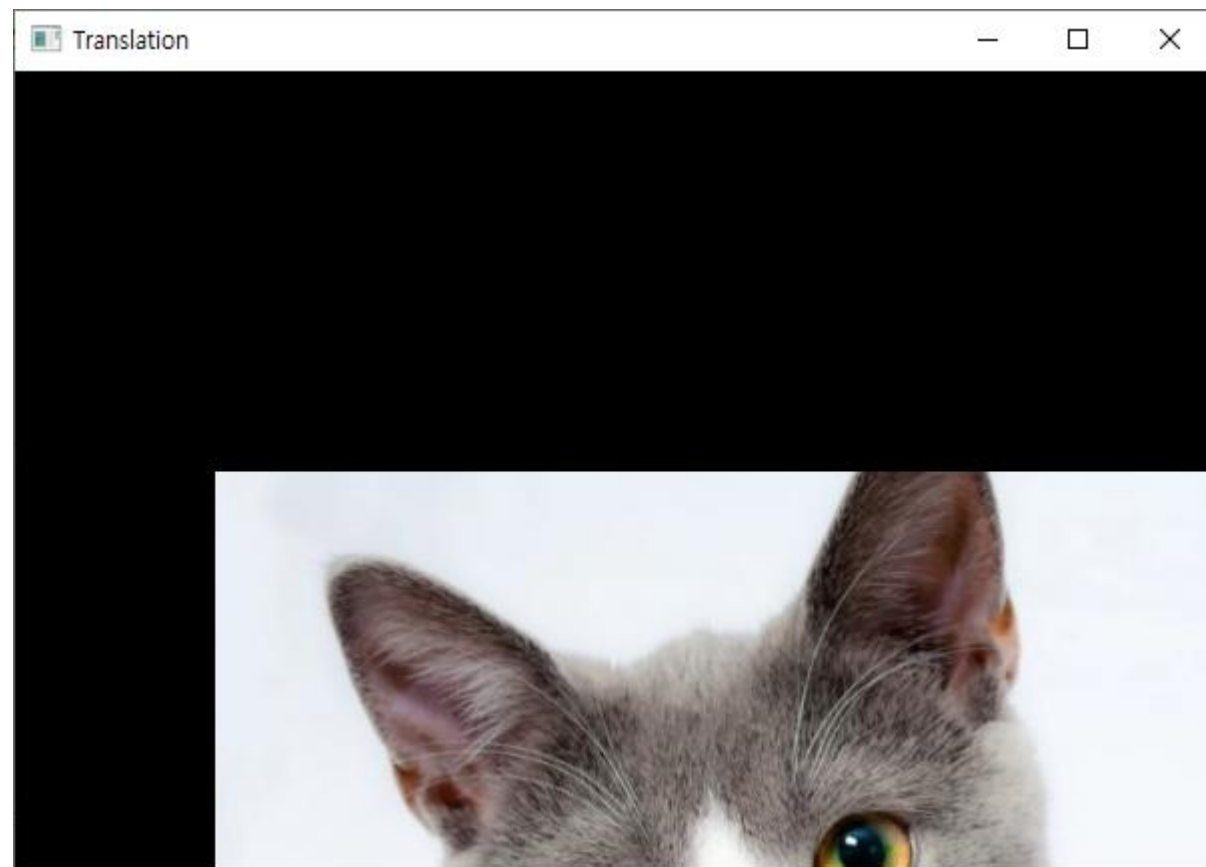
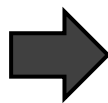
cv2.waitKey(0)
cv2.destroyAllWindows()
```



OpenCV Image Transformations

- 이미지 위치 변경하기(Translation)
- Affine 변환 행렬 생성 → `cv2.warpAffine(src, 변환행렬, dsize)` 으로 변환 적용

```
# 변환 행렬, X축으로 100, Y축으로 200 이동  
M = np.float32([[1,0,100],[0,1,200]])  
  
dst = cv2.warpAffine(img, M,(width, height))  
cv2.imshow('Original', img)  
cv2.imshow('Translation', dst)  
  
cv2.waitKey(0)  
cv2.destroyAllWindows()
```

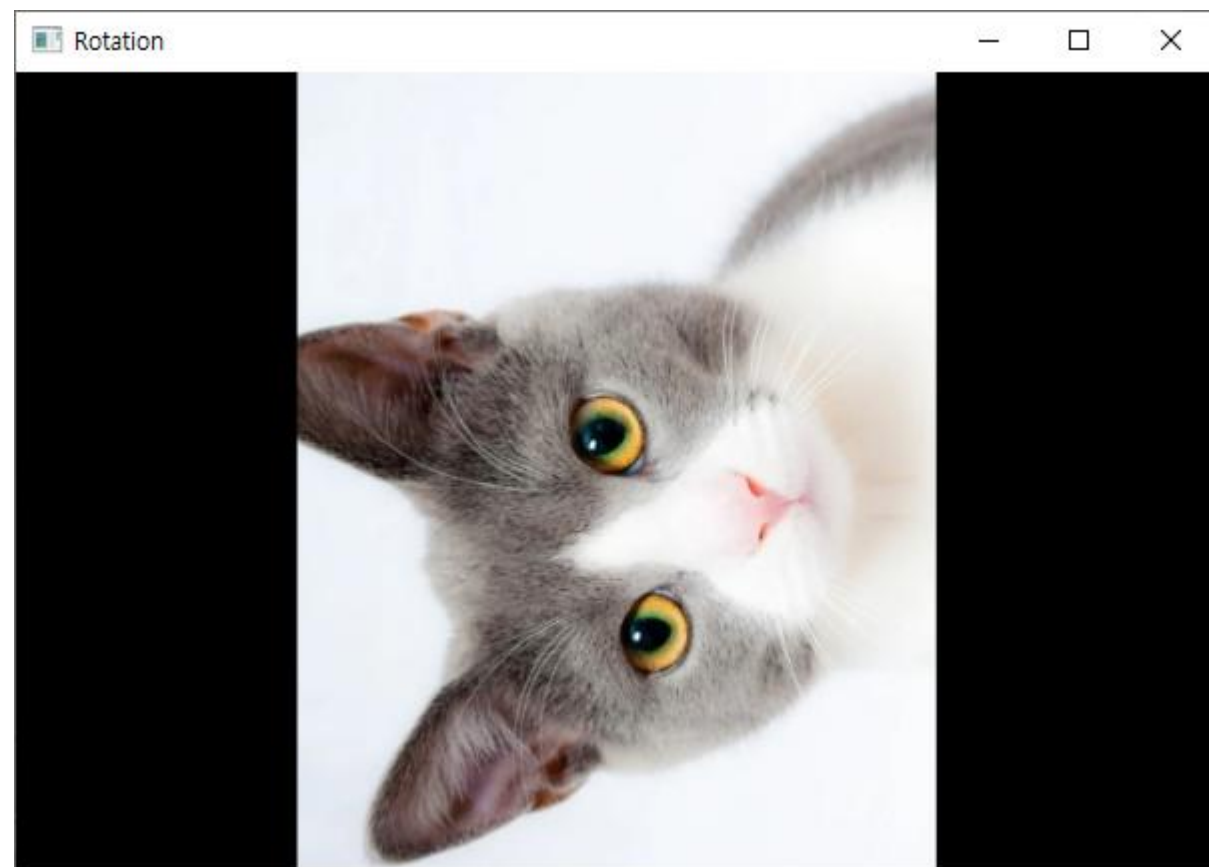
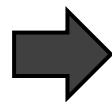


OpenCV Image Transformations

- 이미지 회전하기(Rotation)
- `cv2.getRotationMatrix2D(center, angle, scale)`
→ 해당 center, angle, scale의 Affine 변환 행렬 생성

```
M = cv2.getRotationMatrix2D((width/2, height/2), 90, 0.8)
dst = cv2.warpAffine(img, M, (width, height))

cv2.imshow('Rotation', dst)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



OpenCV Image Transformations

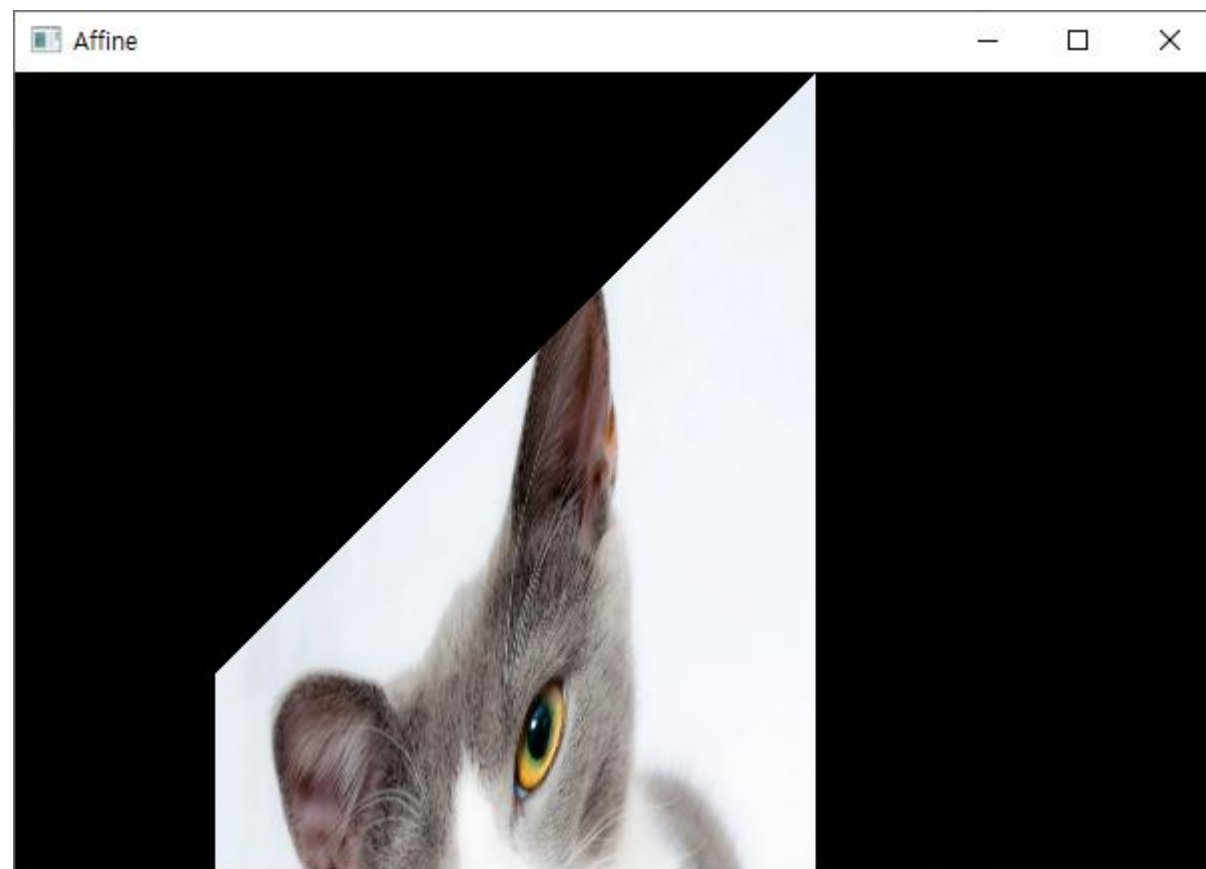
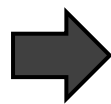
- 3개의 대응점을 활용해 이미지 Affine 변환
- `cv2.getAffineTransform` 함수에 대응점 입력 → 변환 행렬 생성
- `Pts1[[x1,y1], [x2,y2], [x3,y3]] → pts2 [[x1',y1'], [x2',y2'], [x3',y3']]`

```
img = imgcv.copy()
rows, cols, _ = img.shape

pts1 = np.float32([[200,100],[400,100],[200,200]])
pts2 = np.float32([[200,300],[300,200],[200,400]])

M = cv2.getAffineTransform(pts1, pts2)
dst = cv2.warpAffine(img, M, (cols,rows))

cv2.imshow('Affine', dst)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



OpenCV Image Transformations

- 이미지 Perspective 변환하기(4개의 match point사용)
- [x,y] 좌표로 4*2 행렬 구성, L upper – L lower – R upper – R lower

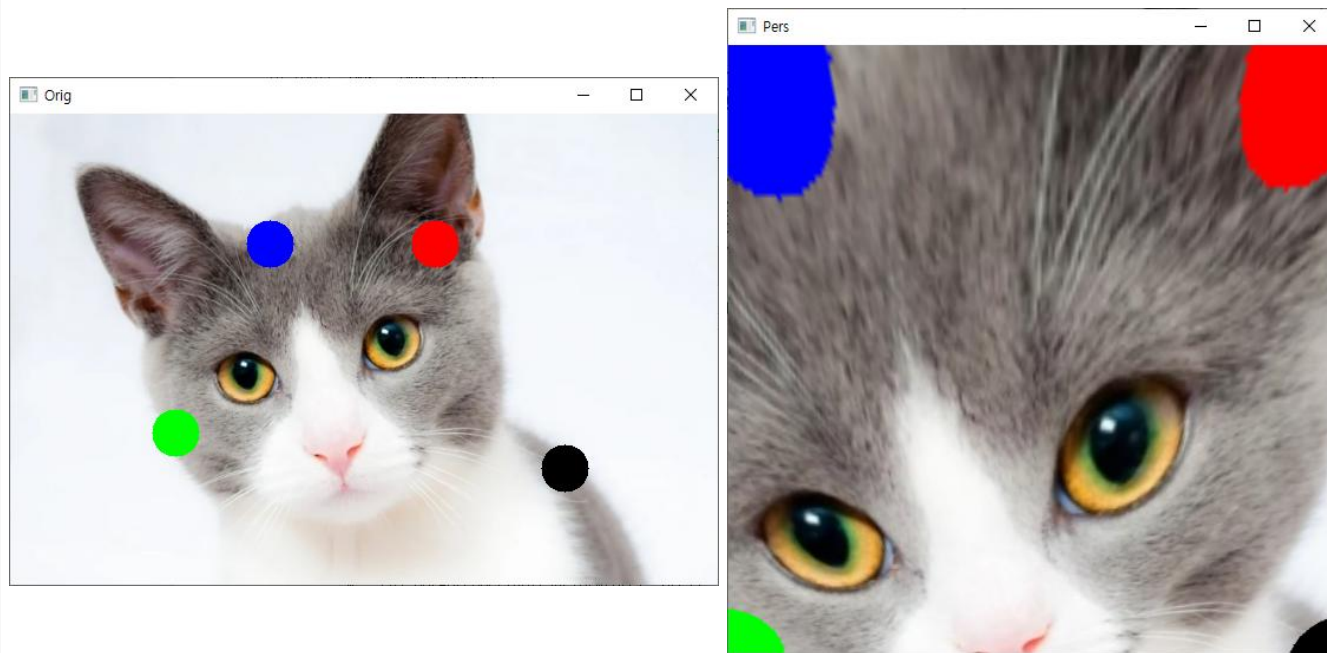
```
img = imgcv.copy()

# 좌표점은 좌상->좌하->우상->우하
pts1 = np.float32([[220,110],[140,270],[360,110],[470,300]])
pts2 = np.float32([[10,10],[10,500],[500,10],[500,500]])

# pts1의 좌표에 표시
cv2.circle(img, (220,110), 20, (255,0,0),-1)
cv2.circle(img, (140,270), 20, (0,255,0),-1)
cv2.circle(img, (360,110), 20, (0,0,255),-1)
cv2.circle(img, (470,300), 20, (0,0,0),-1)

M = cv2.getPerspectiveTransform(pts1, pts2)
dst = cv2.warpPerspective(img, M, (510,510))

cv2.imshow('Pers', dst)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



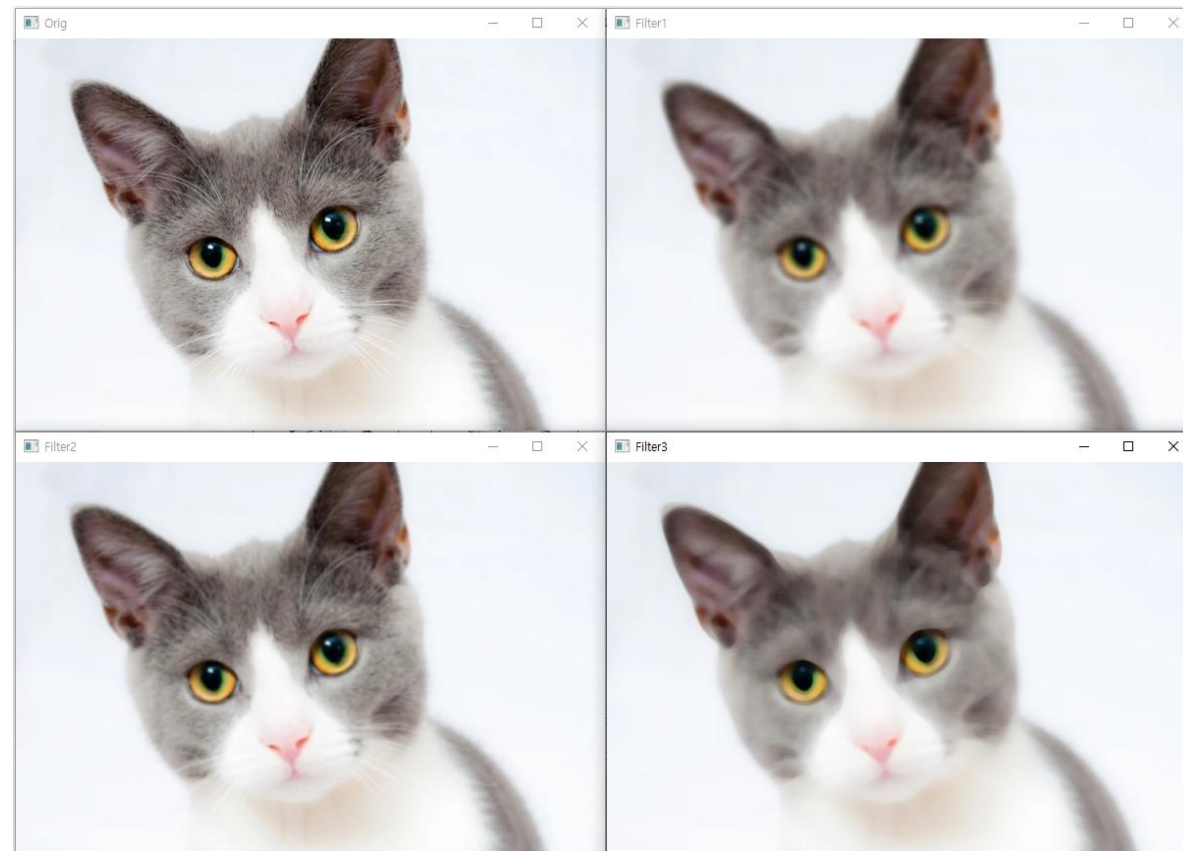
OpenCV Image Filtering

- `cv2.blur(src, ksize)`
- `cv2.GaussianBlur(img, ksize, sigmaX (0이면 auto))`
- `cv2.medianBlur(src, ksize)`: ksize 크기의 사각형 window 안의 중간값

```
img = imgcv.copy()

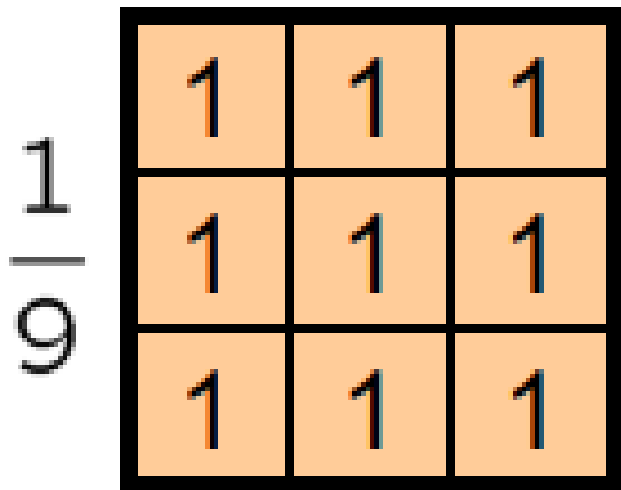
dst1 = cv2.blur(img, (7, 7))
dst2 = cv2.GaussianBlur(img, (5, 5), 0)
dst3 = cv2.medianBlur(img, 9)

images = [img, dst1, dst2, dst3]
cv2.imshow('Orig', img)
cv2.imshow('Filter1', dst1)
cv2.imshow('Filter2', dst2)
cv2.imshow('Filter3', dst3)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



OpenCV Image Filtering

- 직접 filter 지정해서 넣는 경우: `cv2.filter2D(img, ddepth (데이터 타입, -1이면 입력과 동일), kernel)`

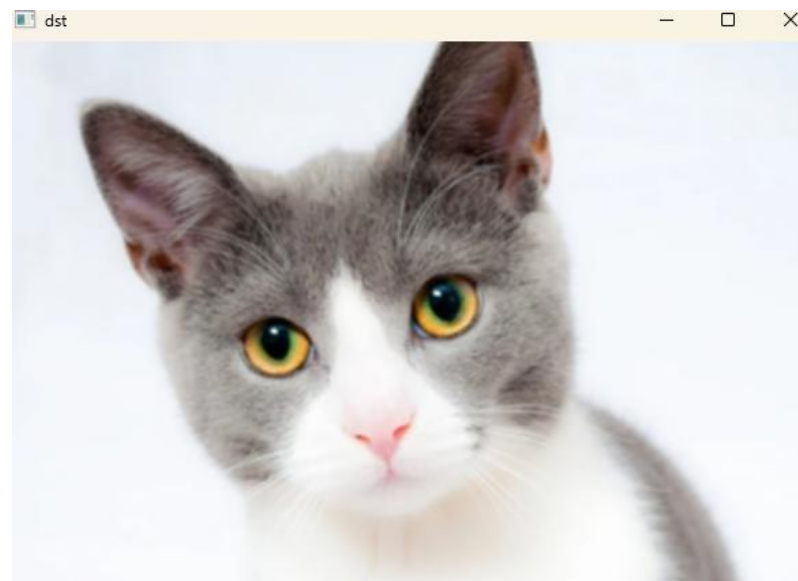


Custom 커널

```
img = imgcv.copy()

kernel = np.ones((3, 3)) / 9.
dst = cv2.filter2D(img, -1, kernel) # -1은 입력, 출력 크기 맞추기

cv2.imshow('dst', dst)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



OpenCV Image Filtering

- 다양한 형태의 커널 지정 가능

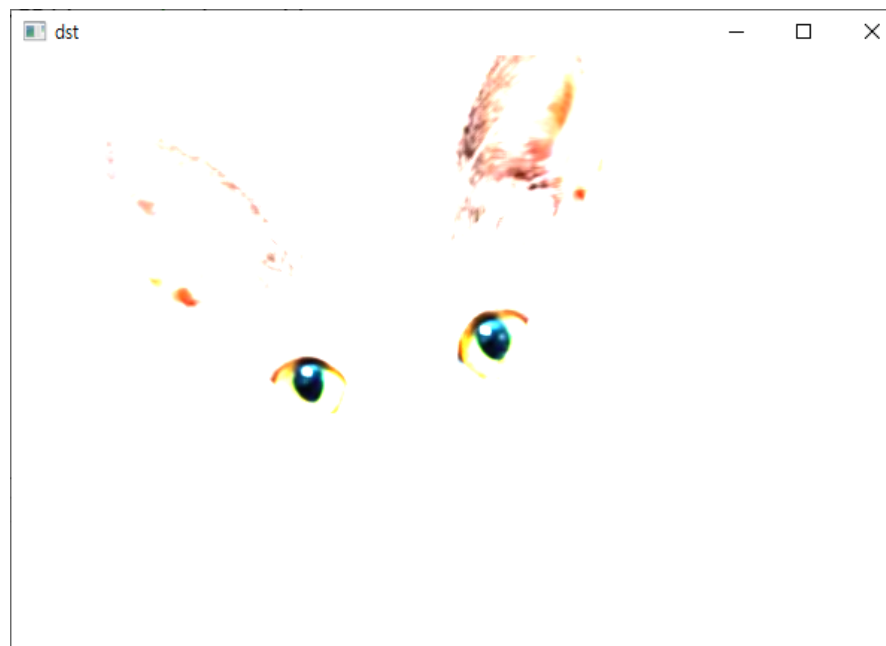
1	0	1
0	1	0
1	0	1

Custom 커널

```
img = imgcv.copy()

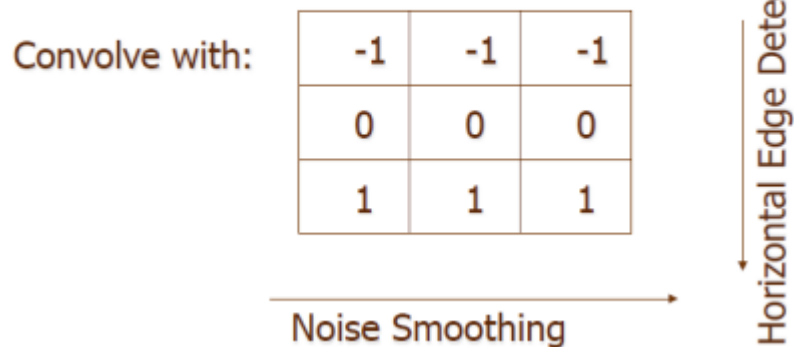
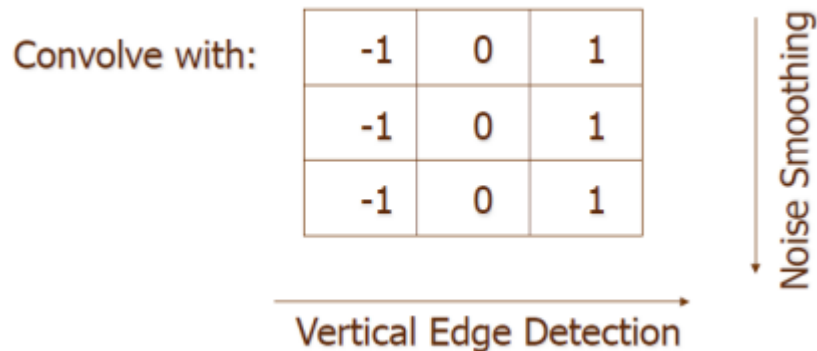
kernel = np.array([[1,0,1], [0,1,0], [1,0,1]])
dst = cv2.filter2D(img, -1, kernel)

cv2.imshow('dst', dst)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



OpenCV Image Filtering

- `cv2.filter2D()`에 커스텀 커널을 지정해 edge detection 수행



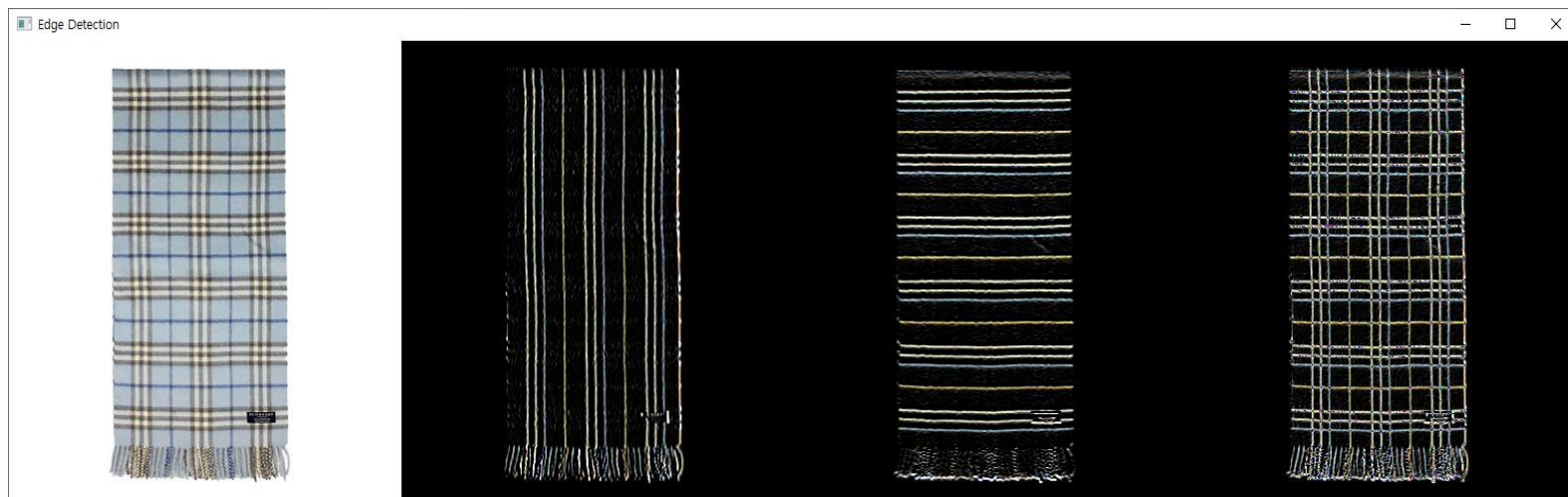
```
img = cv2.imread('edge.jpg')

kernel_ver = np.array([[ -1, 0, 1], [ -1, 0, 1], [ -1, 0, 1]])
kernel_hor = np.array([[ -1, -1, -1], [ 0, 0, 0], [ 1, 1, 1]])

dst_v = cv2.filter2D(img, -1, kernel_ver)
dst_h = cv2.filter2D(img, -1, kernel_hor)

merged = np.hstack((img, dst_v, dst_h, dst_v+dst_h))
cv2.imshow('Edge Detection', merged)
cv2.waitKey(0)
cv2.destroyAllWindows()

titles = ['Original', 'Ver', 'Hor', 'Sum']
images = [img, dst_v, dst_h, dst_v+dst_h]
for i in range(4):
    plt.subplot(2,2,i+1),plt.imshow(cv2.cvtColor(images[i], cv2.COLOR_BGR2RGB))
    plt.title(titles[i])
    plt.xticks([],plt.yticks([]))
```

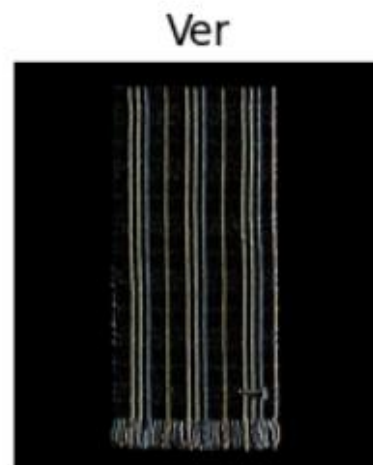
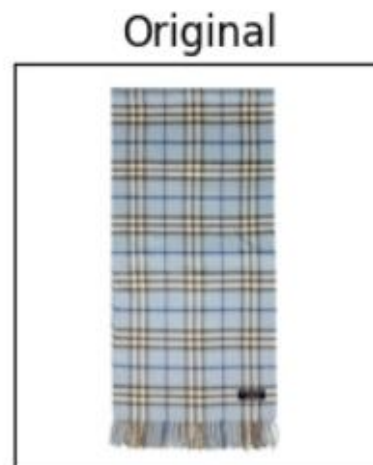


OpenCV Image Filtering

- cv2.Sobel()을 이용한 Edge detection

```
#cv2.Sobel(img, ddepth, dx, dy, kernelsize)
sobelx = cv2.Sobel(img, -1, 1, 0, ksize=3) # x direction
sobely = cv2.Sobel(img, -1, 0, 1, ksize=3) #y direction

images = [img,sobely,sobelx,sobely+sobelx]
titles = ['Original','Hor','Ver','Sum']
for i in range(4):
    plt.subplot(2,2,i+1),plt.imshow(cv2.cvtColor(images[i], cv2.COLOR_BGR2RGB))
    plt.title(titles[i])
    plt.xticks([],plt.yticks([]))
```



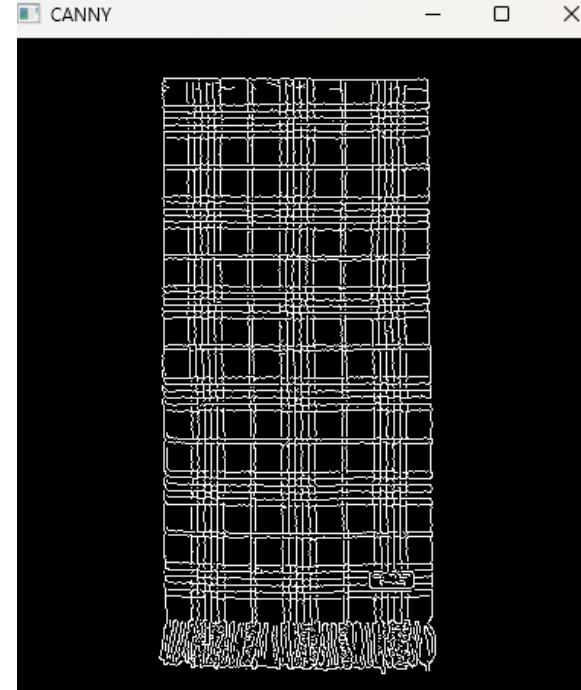
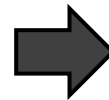
OpenCV Image Filtering

- Canny Edge Detection → multi-stage algorithm

1. Gaussian 필터를 적용하여 노이즈 제거
2. Pixel의 gradient 계산 (magnitude & orientation)
3. Non-maximum Suppression을 적용하여 최대 강도의 gradient direction
4. 실제 edge와 noise들을 Threshold 2개 (min, max)를 이용하여 hysteresis thresholding을 통해 정확한 edge 검출.

`Cv2.Canny(img, threshold1(min), threshold2(max), edges=None, apertureSize=None, L2gradient=None)`

```
canny = cv2.Canny(img, 100, 200)
cv2.imshow("CANNY", canny)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Next Week..

- OpenCV feature, transform
- Image data preparation, scikit-learn

Thank you very much!